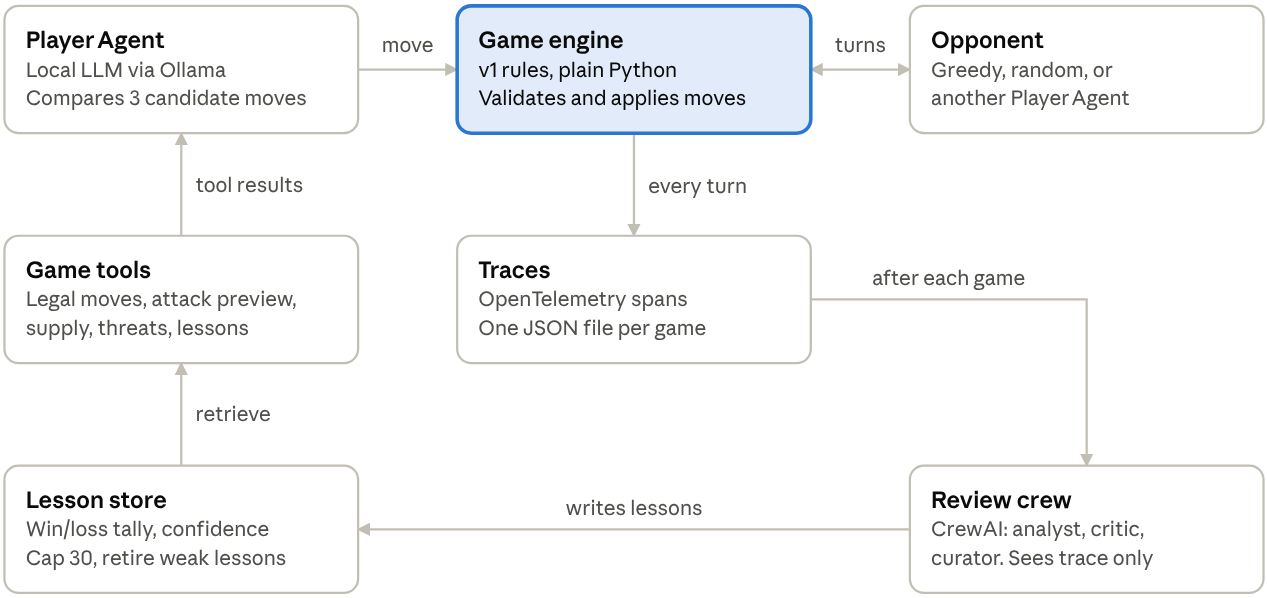
FrontLine Architecture

Oct 11, 2026 · @Chris

FrontLine wraps a deterministic game engine with LLM agents that play, review their own games, and keep only the lessons that win.

At a glance

A plain-code engine at the center, agents learn through a lesson store



system overview · 7 parts, 1 learning loop

Read it as one loop: the Player Agent moves, every turn is traced, the review crew turns traces into lessons, and lessons come back to the player through the lesson-search tool.

Components

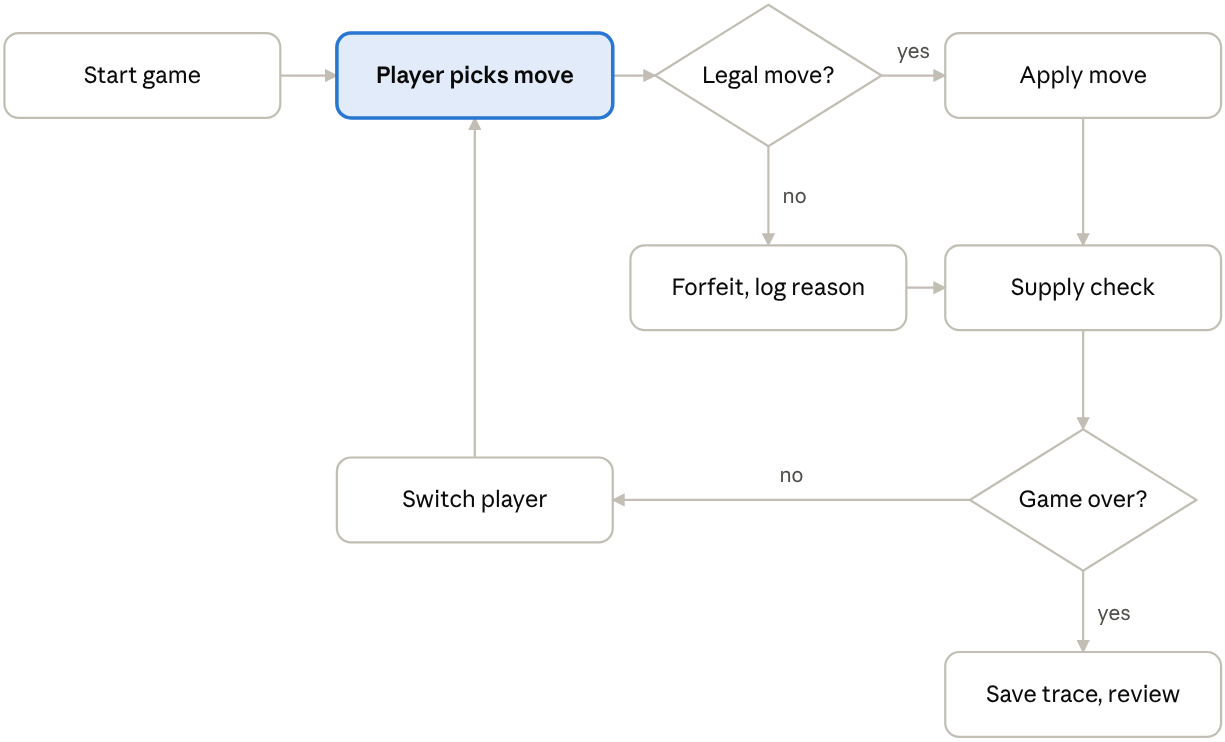
The game engine is plain, deterministic Python. Everything that uses an LLM sits around it, so a bad move can always be traced to the agent, not the rules.

Component	What it does	Tech	Module
Game engine	Applies v1 rules, validates moves, runs supply and win checks	Python, OpenSpiel-style method names	2
Baseline bots	Random and greedy opponents to measure against	Python	6
Match loop	Runs each turn: player move, rules check, win check, next player	LangGraph	5
Player Agent	Picks one move per turn using tools, lessons, and tree-of-thought	Local LLM via Ollama, Pydantic output	2, 4
Game tools	Legal moves, attack preview, supply check, threat scan, lesson search. All read-only	Python functions exposed as tools	2
Lesson store	Lessons with a win/loss tally, confidence, cap of 30, and retire rule	JSON or SQLite plus a vector index	3
Review crew	Analyst, critic, and curator. Sees only the trace and outcome	CrewAI	4, 5
Tracing	One span per turn with child spans for retrieve, decide, validate, apply	OpenTelemetry, saved as JSON	6
Experiment runner	Queues conditions and seeds, resumes after a crash, writes a nightly summary	Python under tmux or systemd	6
Replay viewer	Steps through a saved game move by move	Streamlit	7

Match loop

The LLM picks the move; plain code does everything else

One LangGraph node per box. Game over = enemy home captured or all 24 turns played.

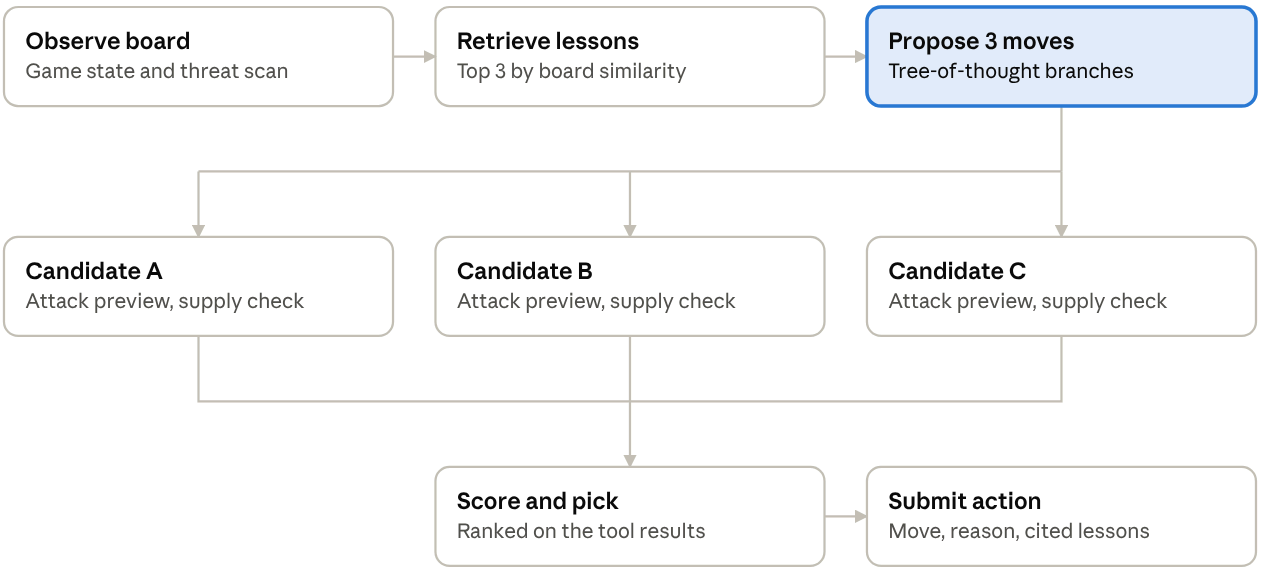


LangGraph match loop · 6 steps, 2 decisions

Only the highlighted step calls an LLM. An illegal move still runs the supply check, because the rules apply supply after every turn.

Player Agent turn

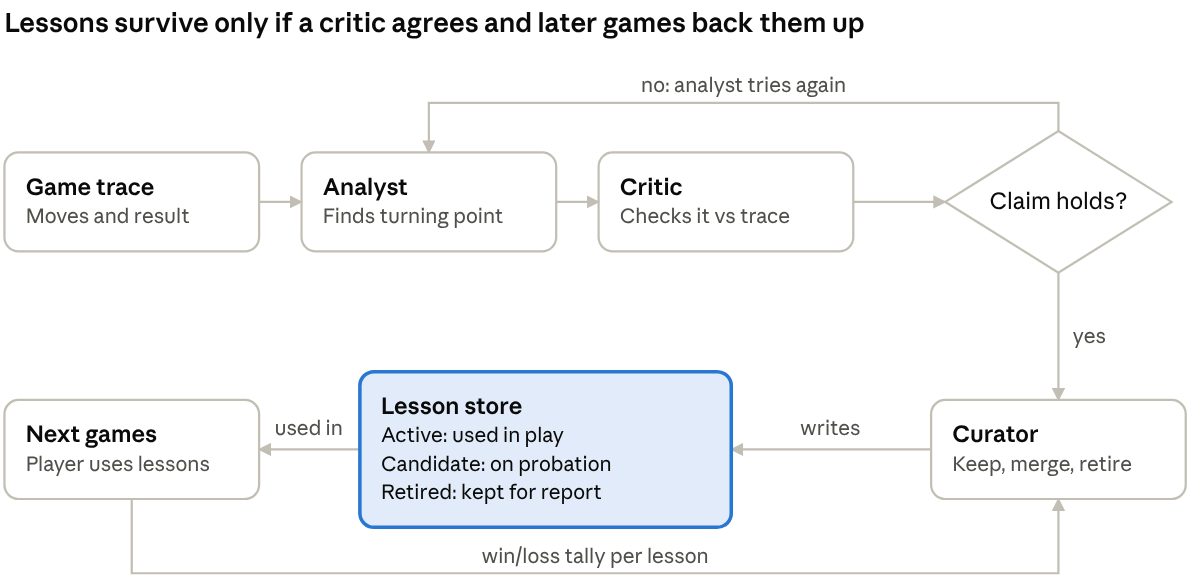
The agent tests three moves with tools before it commits to one



one Player Agent turn · tree-of-thought with 3 branches

The tools do the math, so the LLM never has to calculate attack power or supply in its head. The three candidates and their scores are saved in the trace, which is what lets you show tree-of-thought working in the write-up.

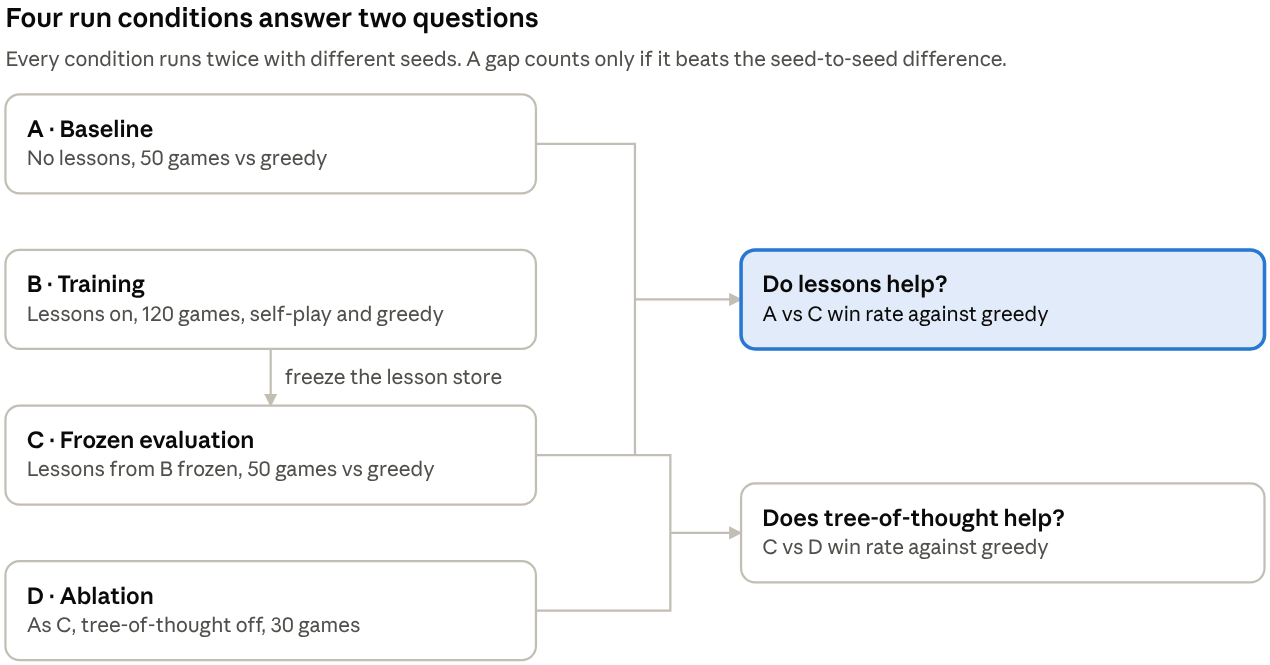
Review crew



CrewAI review crew · 3 roles, 2 checks on every lesson

Each lesson faces two checks: the critic tests it against the trace now, and the win/loss tally tests it in later games. That second check is what separates real learning from an agent quoting advice it ignores.

Experiments



experiment design · 4 conditions, 2 questions

Freezing the store before C matters: it measures what the agent learned, without the agent still learning during the test. The same 4 conditions become the Results slide.

Data contracts

Four schemas connect every part. If these stay stable, you can swap the UI (Streamlit now, Node later) or the model (Ollama or a hosted API) without touching anything else.

```
class Cell(BaseModel):
    owner: Literal["coral", "teal"] | None
    strength: int                    # 0 to 5

class GameState(BaseModel):
    board: dict[str, Cell]           # "A1" to "E5"
    to_move: Literal["coral", "teal"]
    turn: int                        # 1 to 24
    homes: dict[str, str]            # {"coral": "A1", "teal": "E5"}

class Action(BaseModel):
    kind: Literal["claim", "reinforce", "attack"]
```

```
    target: str                         # e.g. "C3"
    source: str | None = None           # attacking cell, attack only
    reason: str
    cited_lessons: list[str] = []

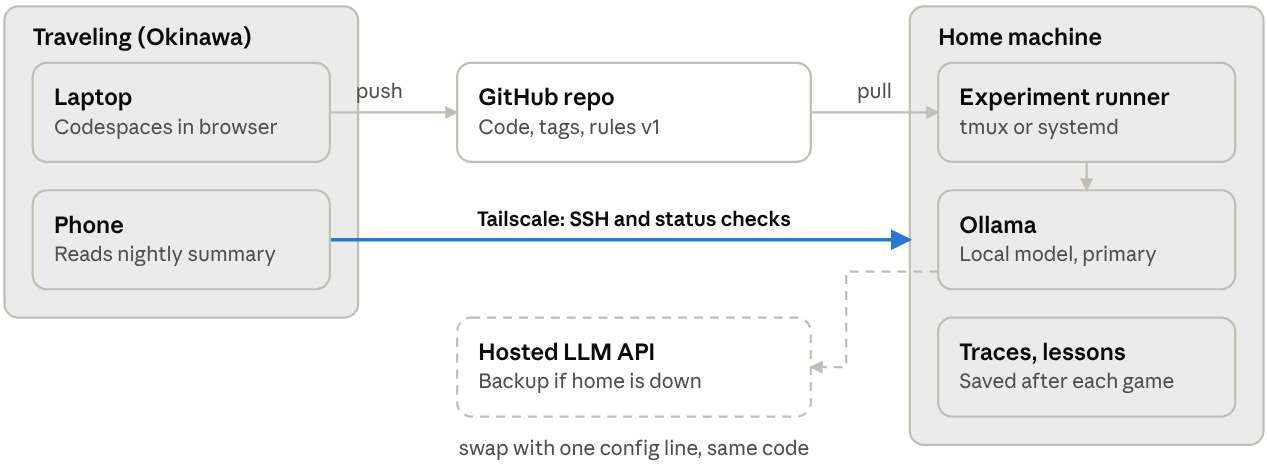
class TurnRecord(BaseModel):
    turn: int
    player: str
    state_before: GameState
    action: Action
    legal: bool
    illegal_reason: str | None
    state_after: GameState
    candidates: list[Action]            # tree-of-thought options

class Lesson(BaseModel):
    id: str
    text: str
    trigger: str                        # board situation it applies to
    wins: int
    losses: int
    confidence: float
    status: Literal["candidate", "active", "retired"]
    source_game: str
```

The review crew gets a stripped copy of each TurnRecord: no `reason`, no `cited_lessons`, no `candidates`. That keeps the reviewer from grading the player's own story.

Deployment

Code lives in GitHub, experiments run at home, Tailscale checks in



deployment during the Okinawa trip · home runs, remote checks

Codespaces is for editing only, because it shuts down when idle. Long runs live on the home machine. If you switch to the hosted API mid-trip, rerun the baseline on that model so results stay comparable.

Slide plan

Each diagram in this doc has a one-line title that states its point, so it can drop straight onto a slide. The deck needs about 10 slides.

#	Slide	Comes from	Visual
1	FrontLine: a strategy research agent	Submission 1, problem and user	Title only
2	Why a plain LLM is not enough	Submission 1, section 2	4 short bullets
3	System at a glance	At a glance	Diagram
4	How a game runs	Match loop	Diagram
5	How the agent picks a move	Player Agent turn	Diagram

#	Slide	Comes from	Visual
6	How the agent learns	Review crew	Diagram
7	How we test it	Experiments	Diagram
8	Results	Analysis notebook, Oct 24	Win-rate charts and results table
9	A lesson changing a move	The 2 best traces	Replay viewer screenshots
10	Limits and next steps	Write-up	AlphaZero comparison, human play, memory poisoning

Slides 8 and 9 stay empty until the experiments run.